Research Computing Documentation

Overview ▼

Tutorials ▲

- **ColdFront Tutorial (coldfront_tutorial.html)**
- **Linux & Bash Tutorial (bash_tutorial.html)**
- **SSH Tutorial (ssh_tutorial.html)**
- **Slurm Tutorial 1: Getting Started (slurm_quick_start_tutorial.html)**
- **Slurm Tutorial 2: Scaling Up (slurm_tutorial_2.html)**
- **Software Tutorial (software_tutorial.html)**
- **Storage Tutorial (storage_tutorial.html)**
- **Globus Tutorial (globus_tutorial.html)**

Quick Reference ▼

Misc Instructions ▼

Resources & Links ▼

Slurm Tutorial 1: Getting Started

Table of Contents

- 0 - Prerequisites
- 1 - Getting Started with Slurm
  - 1.1 - Configuration Options for Slurm
  - 1.2 - Example Slurm Script
  - 1.3 - Submitting Your Job
  - 1.4 - Monitoring Your Job
  - 1.5 - Debugging Your Job
  - 1.6 - Maintenance Windows
- 2 - Resource Selection
  - 2.1 - Different Resource Scenarios
  - 2.2 - Job Scheduling
  - 2.3 - Determining What Resources Your Job Needs
- 3 - Interactive Jobs
  - 3.1.1 - Caveats for Windows

- 4 - Slurm Command Reference
  - 4.1 - sinfo
  - 4.2 - squeue
  - 4.3 - sbatch
  - 4.4 - scancel
  - 4.5 - sacct
  - 4.6 - scontrol
  - 4.7 - my-accounts
  - 4.8 - sinteractive
  - 4.9 - sprio
- 5 - Other Ways to Submit Jobs
  - 5.1 - Open OnDemand
- 6 - Conclusions
- Slurm Tutorial Part 2: Scaling Up (slurm_tutorial_2.html)

On a compute cluster, there are often many people competing to use a finite set of resources (e.g. CPUs, GPUs, RAM). If everyone on the cluster just starts running code, then everyone will have a bad time as resources get shared between all of the different programs running. To solve this problem, Research Computing uses a resource manager and job scheduler called Slurm.

With Slurm, you can submit your compute job and tell Slurm what resources you need. Slurm will allocate those resources to your job and your job alone (so you won't have to share) and then schedule your job. By using Slurm, you are ensuring that the resources you request are only being used by you, which means your code will run more efficiently and won't have any impact on other researchers.

In this tutorial, we will cover how to determine what resources you need, how to tell Slurm what resources you need, how to submit your compute jobs, and how to monitor your compute jobs.

0 - Prerequisites

If you are not familiar with the command line on Linux or with bash scripting, we **strongly** recommend you go through the Linux & Bash Tutorial (bash_tutorial.html) before this Slurm tutorial.

1 - Getting Started with Slurm

To tell Slurm what resources you need, you will have to create an **sbatch** script (also called a Slurm script). In this tutorial, we will be writing sbatch scripts with bash, but you can use any programming language as long as the pound sign (`#`) doesn't cause an error. Your sbatch scripts will generally follow this format:

```
#!/bin/bash -l
# Declaring Slurm Configuration Options

# Loading Software/Libraries

# Running Code
```

Note: You may recall from our **Linux & Bash Tutorial (bash_tutorial.html)** the line `#!/bin/bash -l` above. This line tells our terminal what program to run this file with. In this case, bash.

Let's start by going over the different configuration options for Slurm.

1.1 - Configuration Options for Slurm

There are many configuration options available to Slurm. Some options can be confusing, others may not be able to work together, and some options may have unintended side-effects. You can see a full list of options here, but we recommend you stick with the basics or ask for help if you need to use more advanced options. We can help you find the best set of configurations for your compute needs.

Configuration options are specified in your sbatch script like this:

```
#SBATCH <option_1>=<value>
#SBATCH <option_2>=<value>
...
#SBATCH <option_N>=<value>
```

Note that the pound sign (`#`) is not a comment here. Slurm looks for lines starting with `#SBATCH` so it can find configuration options.

Note: In this tutorial, we will writing sbatch scripts in bash, but you can write an sbatch script in any language as long as `#SBATCH` doesn't result in errors in your language of choice. Examples: Ruby, Python, Bash, R.

1.1.1 - Accounting Configurations

1. **Job Name:** `#SBATCH --job-name=<job_name>`
    - The first thing you will want to do is give your job a name. It should be descriptive, but succinct. Example: `#SBATCH --job-name=LogisticRegression` .
    - The point of the job name is to remind yourself what you are doing. If it's not descriptive, then you can easily get confused.
2. **Comment:** `#SBATCH --comment=<comment>`

- If you want an extended description for your job, you can add a comment. Example: `#SBATCH --comment="Logistic Regression with L2 penalty and liblinear solver."`

3. **Account:** `#SBATCH --account=<account_name>`
    - You need to tell Slurm which account to run your job under. This is *not* your user account, but your project account, which was assigned when you filled out the questionnaire. Example: `#SBATCH --account=cosmos` (we like to have fun with account names).
    - If you don't remember your account name, you can run `my-accounts` on the cluster to find it.
4. **Partition:** `#SBATCH --partition=<debug,tier3>`
    - Slurm needs to know which partition to run your job on. A partition is just a group of nodes (computers). We have three partitions: `debug`, `tier3`, and `interactive`. Each partition has access to different resources and has a specific use case. Example: `#SBATCH --partition=debug`.
    - The `debug` partition is for debugging your code/sbatch script and getting your compute job to run. It should only be used for debugging. **DO NOT** run actual research jobs on the `debug` partition.
    - Once you are finished debugging, you should run your research jobs on the `tier3` partition. Jobs running on `tier3` will not be canceled (unless there are extreme circumstances).
    - The `interactive` partition is for interactive jobs. We will talk more about interactive jobs later in this tutorial.
5. **Time Limit:** `#SBATCH --time=D-HH:MM:SS`
    - You need to tell Slurm how long your job needs to run. The format is Days-Hours:Minutes:Seconds (`D-HH:MM:SS`). Example: `#SBATCH --time=1-12:30:00` (1 day, 12 Hours, 30 Minutes, 0 Seconds).
    - The `tier3` partition has a max time limit of 20 days. If you try to specify more than 20 days on `tier3`, Slurm will not schedule your job.
    - The `debug` partition has a max time limit of 1 day.
    - It's okay to specify a bit more time than you think your job needs. It will not be a good day if your job took 3 days to start running, another 4 days to actually run, and then you find out it actually needs 4 days and 1 minute to finish.

Warning: The `debug` partition is for debugging ONLY. We reserve the right to cancel jobs running on the `debug` partition if we need to train a new researcher or help a researcher debug their jobs. DO NOT run production jobs on the `debug` partition.

Note: It's okay to specify a bit more time than you think your job needs. It will not be a good day if your job took 3 days to start running, another 4 days to actually run, and then your find out it actually

needs 4 days and 1 minute to finish.

1.1.2 - Job Output Configurations

1. **Output File:** `#SBATCH --output=%x_%j.out`
    - Any output from your compute job will be saved to the output file that your specify.
    - `%x` is a variable that fills in your job name. `%j` is a variable that fills in your job ID number.
    - You can place your output file in a folder (e.g. `#SBATCH --output=logs/%x_%j.out`).
2. **Error File:** `#SBATCH --error=%x_%j.err`
    - Any errors from your compute job will be saved to the error file that your specify.
    - `%x` is a variable that fills in your job name. `%j` is a variable that fills in your job ID number.
    - You can place your error file in a folder (e.g. `#SBATCH --error=logs/%x_%j.err`).

Warning: If you place your output or error files in a folder, you need to make sure that folder exists before you submit your job. Otherwise, those files may not get saved, or your job may not even schedule.

1.1.3 - Slack Configurations

1. **Slack Username:** `#SBATCH --mail-user=slack:@<your_username>`
    - You can receive slack notifications from Slurm about your compute jobs. Example: `#SBATCH --mail-user=slack:@abc1234`.
    - You must be logged into slack to receive the notifications
2. **Notification Type:** `#SBATCH --mail-type=<BEGIN,END,FAIL,ALL>`
    - You can tell Slurm what kinds of slack notifications you want to receive. The options are `BEGIN` (when your job starts), `END` (when your job finishes), `FAIL` (if your job fails), and `ALL` (all of the previous conditions).

Warning: This feature overrides Slurm's default email notifications, which do not work in our environment.

Warning: If you submit a large number of jobs, make sure they **do not** send slack notifications. Sending too many notifications at once will essentially spam your slack client.

1.1.4 - Node Configurations

A **node** is just a computer in a cluster. Most of the time, it probably makes sense to only use one node, but if your code can leverage MPI (Message Passing Interface), then your job will probably schedule faster on multiple nodes. If you're unsure how many nodes you need, we can help you figure that out.

1. **Nodes:** `#SBATCH --nodes=<num_nodes>`
    - Example: `#SBATCH --nodes=1`
    - The default is 1 node, so if you're only using 1 node, you don't need to include this configuration option. However, we recommend that you still include it to help remind yourself what resources your job is using.
2. **Excluding Nodes:** `#SBATCH --exclude=<node1,node2,...>`
    - If for some reason you want to make sure your job does not run on a specific node (or nodes), you can do that with this option. Example: `#SBATCH --exclude=theocho`.
3. **Exclusive Access to a Node:** `#SBATCH --exclusive`
    - If your job can fully utilize all of the resources on a single node, then you should specify `#SBATCH --exclusive` to get exclusive access to a whole node. If you're not sure if your job can benefit from this configuration option, we can help you figure that out.

1.1.5 - Task Configurations

In the context of Slurm, a **task** is a running instance of a program. In most situations, you can think of tasks as equivalent to **processes**.

1. **Number of Tasks:** `#SBATCH --ntasks=<num_tasks>`
    - By default, Slurm will assign one task per node. If you want more, you can specify that with this configuration options. Example: `#SBATCH --ntasks=2`.
2. **Number of Tasks per Node:** `#SBATCH --ntasks-per-node=<num_tasks>`
    - If your job is using multiple nodes, you can specify a number of tasks per node with this option. Example: `#SBATCH --ntasks-per-node=2`.

1.1.6 - CPU & GPU Configurations

1. **CPUs per Task:** `#SBATCH --cpus-per-task=<num_cpus>`
    - Slurm needs to know how many CPUs your job needs. Example: `#SBATCH --cpus-per-task=4`.
    - By default, Slurm will assign 1 CPU per task if you do not use the configuration option.
2. **GPUs per Job:** `#SBATCH --gres=gpu:<gpu_type>:<num_gpus>`
    - By default, Slurm will not allocate any GPUs to your job. You need to specify how many and what type of GPUs your job needs.
    - We have Nvidia's a100's available. GPUs are a hot commodity, so make sure that the GPUs you request are actually being used.

- Example: `#SBATCH --gres=gpu:a100:1` .

3. **GPUs per Task:** `#SBATCH --gpus-per-task=<gpu_type>:<num_gpus>`
    - How many GPUs to allocate per task.
    - You can use this in conjunction with `#SBATCH --gres` or on it's own.
    - Example: `#SBATCH --ntasks=2 --gpus-per-task=a100:1` will request 1 a100 per task, so 2 a100's total.

Warning: We have a limited number of GPUs and everyone wants to use them. It's important to make sure that the GPUs you request are actually being used by your code. If you have idle GPUs, no one else can use them until your job finishes running. We reserve the right to cancel jobs with idle GPUs.

1.1.7 - Memory Configurations

1. **Memory per Node:** `#SBATCH --mem=<memory>`
    - You can use this option to tell Slurm how much memory you need per node. Example: `#SBATCH --mem=10g` (10GB of memory per node).
    - The default is megabytes (MB), so if you just say `#SBATCH --mem=10` , you will only get 10MB. You can use `k` for kilobytes (KB), `m` for megabytes (MB), `g` for gigabytes (GB), and `t` for terabytes (TB).
2. **Memory per CPU:** `#SBATCH --mem-per-cpu=<memory>`
    - You can also specify a memory limit per CPU. Example: `#SBATCH --mem-per-cpu=10g` (10GB of memory per CPU).
    - You need to make sure `--mem` and `--mem-per-cpu` don't conflict with each other. In the following example, we ask for 2 nodes with 1 task each, and 2 CPUs per task (4 CPUs total). We also ask for 20GB of memory per node. Since each node only has 20GB of memory and 2 CPUs, the maximum memory we can request per CPU is 10GB. Slurm will not schedule the following example because we are asking for too much memory per CPU.

```
...
#SBATCH --nodes=2
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=2
#SBATCH --mem=20g
#SBATCH --mem-per-cpu=20g
...
```

3. **All Memory On One Node:** `#SBATCH --mem=0`
    - If you are using `#SBATCH --exclusive` , you should set `#SBATCH --mem=0` to tell Slurm to allocate all of the memory on your node.

Warning: The default size for memory is Megabytes (MB). If you just say `#SBATCH --mem=10`, for example, Slurm will only allocate 10MB. You can use `k` for Kilobytes (KB), `m` for Megabytes (MB), `g` for Gigbytes (GB), and `t` for Terabytes (TB).

1.1.8 - Slurm Filename Variables

You saw with the `--output` and `--error` configuration options that we used two special variables: `%x` (job name) and `%j` (job ID). Slurm provides a number of variables for you to use for naming files. Here are a few that you may find useful:

Variable	Example	Description
`%x`	`#SBATCH --output=%x_%j.out`	Fill in job name (set by `--job-name`)
`%j`	`#SBATCH --error=%x_%j.err`	Fill in job ID (set by Slurm)
`%N`	`#SBATCH --output=%n_%x_%j.out`	Fill in hostname; creates separate file for each host
`%a`	`#SBATCH --error=%x_%a_%j.err`	Fill in job array number (job arrays covered in Part 2 of this tutorial)
`%%`	`#SBATCH --output=%x_20%%.out`	Escape percent sign; creates `<job_name>_20%.out`

Here is a full list of Slurm filename variables.

1.2 - Example Slurm Script

Now we can combine some of the options above to create a simple job. Let's create an sbatch script (`$ vim test_script.sh`) and place the following code inside of it:

```
#!/bin/bash -l

#SBATCH --job-name=testJob              # Name for your job
#SBATCH --comment="Testing Job"         # Comment for your job

#SBATCH --account=rc-help               # Project account to run your job under
#SBATCH --partition=debug               # Partition to run your job on

#SBATCH --output=%x_%j.out              # Output file
#SBATCH --error=%x_%j.err               # Error file

#SBATCH --mail-user=slack:@abc1234      # Slack username to notify
#SBATCH --mail-type=END                 # Type of slack notifications to send

#SBATCH --time=0-00:05:00               # Time limit
#SBATCH --nodes=1                       # How many nodes to run on
#SBATCH --ntasks=2                      # How many tasks per node
#SBATCH --cpus-per-task=2               # Number of CPUs per task
#SBATCH --mem-per-cpu=10g               # Memory per CPU

hostname                                # Run the command hostname
```

So, in this example, we have requested a job with the following dimensions:

- **Max Run Time:** 5 Minutes
- **Number of Nodes:** 1
- **Number of Tasks Per Node:** 2
- **Number of CPUs Per Task:** 2
- **Memory Per CPU:** 10GB

We have also told Slurm to run on the `debug` partition under the `rc-help` project account, and to send us a slack notification when the job finishes.

Finally, we run the bash command `hostname`. You can run whatever kind of code you want here; C, C++, bash, python, R, Ruby, etc.

1.3 - Submitting Your Job

Submitting your job is easy! Simply use the command `$ sbatch <slurm_script.sh>`. In this example:

```
$ sbatch test_script.sh
Submitted batch job 15289113
```

Notice above that Slurm responded and gave us a job ID. That job ID is unique and you can use it to monitor your job. We can also use it to help you debug if something goes wrong.

1.4 - Monitoring Your Job

After we submit a job, Slurm will create the output and error files. You can see them by running:

```
$ ls
testJob_15289113.out    testJob_15289113.err    test_script.sh
```

We can also see that our job is running using the `squeue --me` command:

```
$ squeue --me
       JOBID PARTITION     NAME     USER  ST       TIME  NODES NODELIST(REASON)
    15311319     debug simple_m   abc1234  R       0:19      1 skl-a-47
```

The `squeue` command gives us the following information:

- **JOBID:** The unique ID for your job.
- **PARTITION:** The partition your job is running on (or scheduled to run on).
- **NAME:** The name of your job.
- **USER:** The username for whomever submitted the job.
- **ST:** The status of the job. The typical status codes you may see are:
    - `CD` (Completed): Job completed successfully
    - `CG` (Completing): Job is finishing, Slurm is cleaning up
    - `PD` (Pending): Job is scheduled, but the requested resources aren't available yet
    - `R` (Running): Job is actively running
- **TIME:** How long your job has been running.
- **NODES:** How many nodes your job is using.
- **NODELIST(REASON):** Which nodes your job is running on (or scheduled to run on). If your job is not running yet, you will also see one of the following reason codes:
    - `Priority` : When Slurm schedules a job, it takes into consideration how frequently you submit jobs. If you often submit many jobs, Slurm will assign you a lower priority than someone who has never submitted a job or submits jobs very infrequently. Don't worry, your job will run eventually.
    - `Resources` : Slurm is waiting for the requested reasources to be available before starting your job.

- `Dependency` : If you are using dependent jobs, the parent job may show this reason if it's waiting for a dependent job to complete.

> **Note:** Slurm takes into account two different factors when scheduling jobs: **Requested Resources** and **Priority**. If you request a lot of resources, your job may take longer to start than someone who requests very few resources because Slurm needs to wait for the resources you requested to be available. If you are constantly submitting and running jobs, Slurm may assign your jobs a lower priority than someone who rarely submits jobs.

You can also run `squeue` on its own to see all of the jobs Slurm currently has scheduled.

Any output from your job will be written to the output file that you specified (with `#SBATCH --output=%x_%j.out`). You can see the contents of this file using `cat` or `tail -f` . See our Linux & Bash Tutorial (bash_tutorial.html) for details on how to use those commands.

1.5 - Debugging Your Job

If your job fails, you need to examine the output and error files that you specified. The error messages you see will help you decide what you need to do to get your job to run. Often, these errors are specific to the programming language you are using and a quick Google search will help you figure out what went wrong.

Note that some programs (*e.g.* Ansys) will write logs to their own files. If your Slurm output and error files aren't giving you any useful information, look for any new files in your job's working directory that you did not put there.

However, sometimes Slurm may kill your job or decide not to schedule your job because your sbatch script doesn't have the right configuration options. Here are some typical error messages from Slurm:

1.5.1 - Account and Partition Errors

```
sbatch: error: Unable to allocate resources: Invalid account or account/partition combin
ation specified
```

If you see this error, make sure your sbatch script specifies your Slurm account (`#SBATCH --account=<your_account_name>`) and a partition (`#SBATCH --partition=<tier3,debug>`). If you don't have those options, Slurm will not schedule your job.

If you have both of those options, make sure your account has not expired using the `my-accounts` command:

```
$ my-accounts
  Account Name     Expired  QOS              Allowed Partitions
- ------------     -------  ---              ------------------
  swen-331         true     qos_ood          ood
  mistakes         false    qos_tier3        interactive
* rc-help          false    qos_onboard      tier3,debug,onboard,interactive
```

Looking at the output above, you will see that the `swen-331` account is expired, which means Slurm cannot submit jobs where the account is set to `swen-331`. If you believe your account should not be expired, please send us an email.

In the output above, you will also see which partitions are valid for each account. The account `mistakes` is only allowed to run on the `interactive` partition (we will discuss that partition later in this tutorial). So if I try to submit a job with

```
#SBATCH --account=mistakes
#SBATCH --partition=tier3
```

Slurm will not schedule it.

Note: The `*` indicates which account is your default account. If you forget to include `#SBATCH --account=<your_account_name>`, then Slurm will try to use your default account.

```
sbatch: error: Unable to allocate resources: Invalid qos specification
```

If you see this error, the the partition you specified is not allowed to run jobs under the account your specified. Double check your account and partition.

```
sbatch: error: Unable to allocate resources: Invalid partition name specified
```

If you see this error, you most likely have a typo in your partition name.

1.5.2 - Resource Specific Errors

```
sbatch: error: Unable to allocate resources: Requested time limit is invalid (missing or
exceeds some limit)
```

If you see this error, double check that you have specified a time limit for your job (`#SBATCH --time=D-HH:MM:SS`). If you have, then you have requested a time limit that is not valid for your selected partition. Recall that `tier3` has a maximum time limit of 20 days, `debug` has a maximum time limit of 1 day, and `interactive` has a maximum time limit of 12 hours.

```
sbatch: error: Batch job submission failed: Requested node configuration is not availabl
e
```

If you see this error, you most likely have some combination of nodes, CPUs, and memory that is not valid. Keep in mind that the majority of the nodes in the cluster have 36 CPUs and 350GB of memory, so if you request a single node with more than 36 CPUs or more than 350GB of memory, Slurm cannot schedule your job.

```
slurmstepd: error: ... Some of your processes may have been killed by the cgroup out-of-
memory handler.
```

If you see this error, then your job tried to use more memory than the amount you requested and Slurm killed your job. You need to resubmit with more memory. We can help you determine how much memory you need if you're not sure.

```
slurmstepd: error: *** JOB <ID> ON <NODE> CANCELLED AT <TIMESTAMP> DUE TO TIME LIMIT ***
```

If you see this error, then your job did not finish before the reaching the maximum time limit and Slurm killed your job. You need to resubmit with a higher time limit.

1.5.3 - Miscellaneous Slurm Errors

```
sbatch: error: Batch script contains DOS line breaks (\r\n)
sbatch: error: instead of expected UNIX line breaks (\n)
```

If you wrote your sbatch script on a Windows machine, then you need to convert your line breaks to newlines (convert_dos_line_endings_to_unix.html).

1.6 - Maintenance Windows

Our SPORC Cluster has regular maintenance windows (usually the second Tuesday of the month). Maintenance will begin at 8AM EST/EDT on the scheduled day and will typically be complete by 5PM EST/EDT on the same day, but we do have occasional two-day maintenance windows. During maintenance windows `sporcsubmit` will not be available and you will not be able to submit compute jobs.

You can see a list of upcoming maintenance windows by running the `time-until-maintenance` command on the cluster.

Please review our maintenance window (maintenance.html) documentation, particularly the section on **job scheduling and maintenance**.

> **Note:** During maintenance windows, the cluster will not be available and your jobs will not run.

2 - Resource Selection

We like to see our usage statistics as high as they can be. Our ideal setup would be 100% utilization with little wait time. Realistically that won't happen. One of the largest factors that impacts utilization is when a researcher requests resources they don't use.

For example, if you request 16 CPUs, but your code is single-threaded, the other 15 CPUs would sit idle until your job finished. Once resources have been allocated to your job, no one else can use them (even if they are sitting idle) until your job finishes.

> **Warning:** Once resources have been allocated to your job, no one else can use them (even if they are sitting idle) until your job finishes. We reserve the right to kill jobs that are wasting large amounts of resources.

2.1 - Different Resource Scenarios

Here is a summary of different resource utilization scenarios:

- **RAM:**
  - **Request too little:** Job will die when it runs out of RAM.
  - **Request too much:** Lots of RAM will sit idle and no one else can use it.
  - ***Ideal:*** Request slightly more RAM than you need.
  - **Recommendation:** Try to keep idle RAM at less than 10% of the total RAM you requested.
- **CPUs:**

    - **Request too little:** Your job will trip over itself because of kernel scheduling; your job will take a massive performance hit as a result.
    - **Request too much:** Lots or CPUs will sit idle and no one else can use them.
    - ***Ideal:*** Request exactly the number of CPUs that your job can use.
- **GPUs:**
    - **Request too little:** You may not actually see a speedup (due to communication overhead between CPUs and GPUs).
    - **Request too much:** Your code may not be able to use multiple GPUs; idle GPUs cannot be used by anyone else until your job finishes.
    - ***Ideal:*** Request exactly the number of GPUs that your job can use.
    - **Recommendation:** Get your job working with one GPU, and make sure you're actually using the GPU before trying to use more.
- **Time:**
    - **Request too little:** Your job will not finish before the time limit runs out; lots of time will be wasted.
    - **Request too much:** Slurm may give your job a lower priority to let smaller jobs go first. If a maintenance window is coming up, your job may not schedule until after the maintenance window.
    - ***Ideal:*** Request slightly more time than you need, but not too much.

2.2 - Job Scheduling

When you submit your job, Slurm grabs your `#SBATCH` configurations and finds a time and place on the cluster to run your job. There are four things that impact when your job will run:

1. The resources you request
2. The frequency that you submit jobs
3. The other jobs in the queue
4. Maintenance windows (maintenance.html)

Some examples:

- If you request a lot of resources, you will have to wait until those resources are available, which may be a while depending on how many other jobs are in the queue.
- If you submit a lot of jobs with a small amount of resources, they will likely schedule quickly, but your future jobs may have a slightly lower priority than jobs submitted by someone who submits jobs infrequently.
- If you submit a job with a 3 day runtime (`#SBATCH --time=3-00:00:00`), but a maintenance window is scheduled for two days from now, there is no way your job will finish before the maintenance

window, so it won't be scheduled until after the maintenance window.

- If there is a maintenance window 4 days away, and you submit a job with a 2 day runtime, but the queue is full and the earliest the resources you requested will be available is in 3 days, there is no way your job will finish before the maintenance window (`2+3=5`), so your job won't be scheduled until after the maintenance window.

2.2.1 - Caveats for Resource Selection

Most of our nodes have 36 CPUs and 350GB of RAM. If you specify a single node (`#SBATCH --nodes=1`) and more than the max CPUs or RAM on a node, Slurm will keep your job in the queue and wait to schedule it until a node with those resources gets added to the cluster (which may be never).

If your job must run on a single node, then it will usually take a bit longer for the resources to be available for your job. If your job can run on multiple nodes, it's much better for you to not specify `#SBATCH --nodes` at all and just let Slurm figure out the right place to put your job (which may be on a single node or on multiple nodes).

Excluding the interactive partition, most nodes in the cluster have 1-4 GPUs. If you request more than 4 GPUs on `--partition=tier3` , Slurm will keep your job in the queue until a node with more than 4 GPUs gets added to the cluster (which may be never).

We **strongly** recommend that you get your job running with a single GPU before trying to use multiple GPUs. Jobs that request multiple GPUs will typically take longer to schedule because the cluster has a limited supply of GPUs.

2.2.2 - Job Speed

Using GPUs **may or may not** result in a speedup for you job. There are a lot of factors in play when it comes to GPUs:

- Your code needs to be able to use GPUs; not all libraries/languages can leverage GPUs. Make sure you read the documentation for your libraries/frameworks.
- If using multiple GPUs, you want to make sure your code can use GPUs on different nodes (because it will take longer for a single node with multiple GPUs to be available).
- Some code can leverage GPUs, but not in an impactful way; some code just isn't doing enough computation to make it worth the overhead of communicating between CPUs and GPUs; in this situation, you may actually see a slowdown for your job.

Because of the way that CPUs are designed and implemented and how our nodes are set up, Slurm likes the number 9. If your job is running slower than you think it should, try changing the number of CPUs you're requesting to a multiple of 9 (*e.g.* 9, 18, 27, 36). You **may or may not** notice a speedup. Of course, if your code can only use a single CPU, don't request more CPUs than your code can use.

As the number of CPUs you request grows, it gets more likely that your job's processes will be spread across multiple **NUMA** domains. We won't get into what exactly NUMA domains are, but you **may or may not** notice a speedup if you tell Slurm to minimize the number of NUMA domains: `srun --cpu-bind=ldoms <command>`.

> **Warning:** NUMA is a nuanced subject, so if you're not confident in your NUMA knowledge, it's best to **not** use the `--cpu-bind=ldoms` flag. If you think you may see a speedup from using it, come talk to us and we can help.

2.3 - Determining What Resources Your Job Needs

It can be tricky guessing what resources your job needs. Luckily, there are some useful tools for determining what resources your job is actually using.

To get started, submit your job. Choose some reasonable resources to start with.

For this example, I'm going to run some code that uses PyTorch to approximate a *sin* curve. Here's my sbatch script:

```
#!/bin/bash -l

#SBATCH --job-name=torch_sin    # Name of your job
#SBATCH --account=rc-help       # Your Slurm account
#SBATCH --partition=tier3       # Run on tier3
#SBATCH --output=%x_%j.out      # Output file
#SBATCH --error=%x_%j.err       # Error file
#SBATCH --time=0-01:00:00       # 1 hour time limit
#SBATCH --ntasks=1              # 1 tasks (i.e. processes)
#SBATCH --mem=4g                # 4GB RAM
#SBATCH --gres=gpu:a100:1       # 1 a100 GPU

spack load py-torchvision@0.11.3 /n5dyeip
spack load py-scikit-learn@1.0.2 /maoxkvt
spack load py-matplotlib@3.5.1 /brhphdf

python3 pytorch_gpu_example.py
```

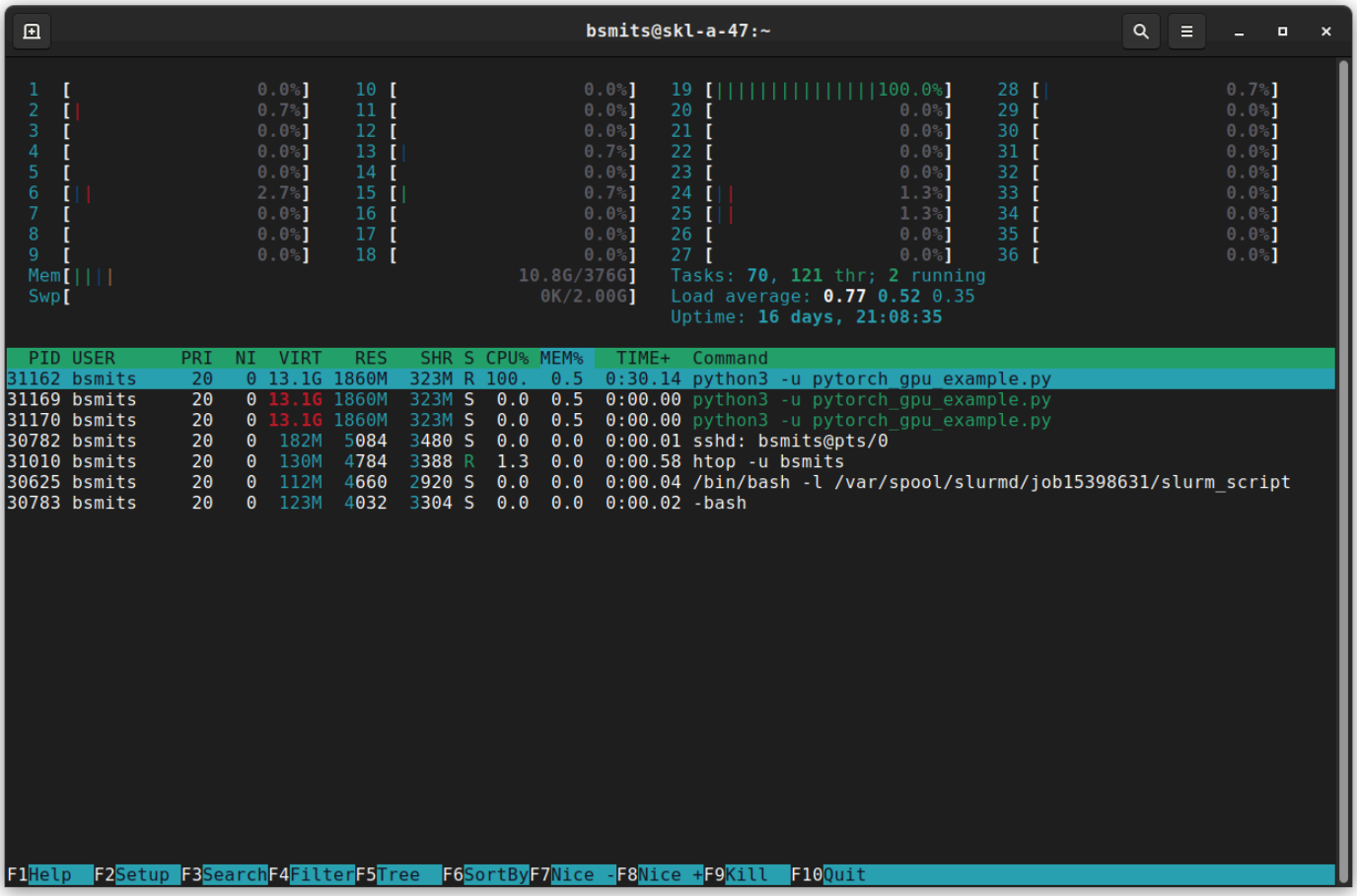
If I submit that script, and then run `squeue --me`, we see the following:

```
$ squeue --me
     JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
  15398629     debug pytorch_   bsmits  R       0:03      1 skl-a-47
```

Now I know that my job is running on the `skl-a-47` node. When you have a job running on a node, you are able to `ssh` into that node, so let's do that: `$ ssh skl-a-47` (no `.rc.rit.edu` needed).

2.2.1 - Monitoring CPU and RAM

On `skl-a-47`, we can run `htop -u <rit_username>`. Htop lets you monitor running processes on a computer. Let's just look at what `htop` looks like:



In the screenshot above, we see two sections (split by the horizontal green row). Above the green row, we see a graph of the CPU usage for all CPUs on `skl-a-47`. Currently, one CPU (#19) is being 100% used (which is what we like to see), and CPUs #2, #6, #15, #24, and #25 are being partially used.

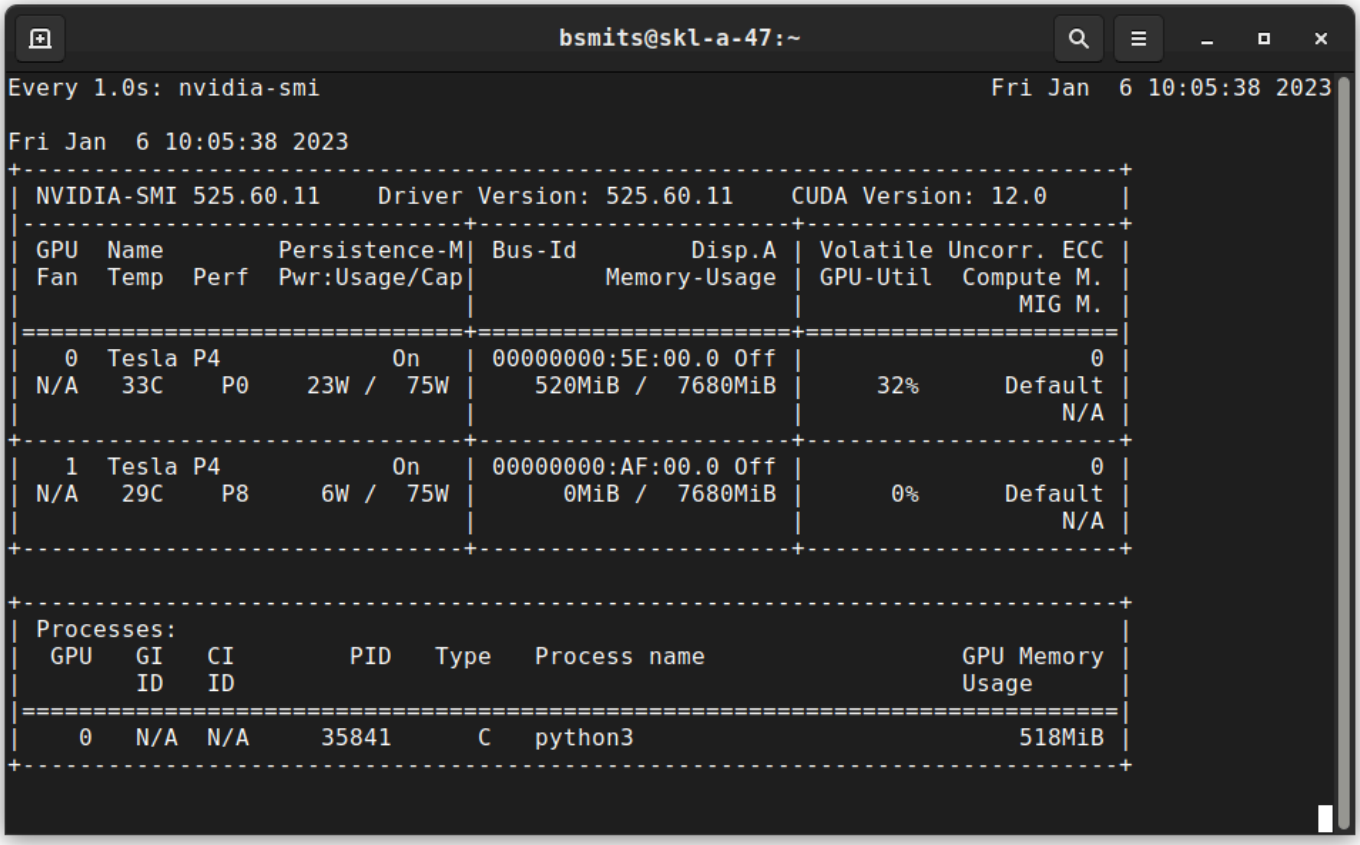
Below the green row, we see all of the processes I am running, notably the `python3` command from my sbatch script. The green row has column names for the processes. `htop` shows us a lot, but right now we only care about the `CPU%` and `MEM%` columns, which show you the CPU and RAM usage, respectively. Right now, our Slurm job is using 100% CPU and 0.5% memory.

Now, the `CPU%` and `MEM%` columns are going to change frequently depending on what your code is doing. If our job is using 100% memory, we should modify our sbatch script to choose *slightly more* memory than we need – if your job tries to use more memory than you asked for, Slurm will kill your job, and no one wants that to happen. If our job is using 100% CPU, then we can *probably* use parallel processing (see Part 2 (slurm_tutorial_2.html) of this tutorial) to speed up our computation.

Note: You should use `htop` to monitor your job's CPU and RAM usage. 100% CPU? Resubmit with more CPUs and see if that speeds up your job (but make sure to check `htop` so you know whether your code is using the extra CPUs or not). 100% RAM? Resubmit with *slightly more* than you need so Slurm doesn't kill your job.

2.2.2 - Monitoring GPU

But CPU and RAM are not the only things we care about, we also want to see what the GPU we asked for is doing. For that, we would run `watch -n 1 nvidia-smi` from `skl-a-47`, which shows us this:

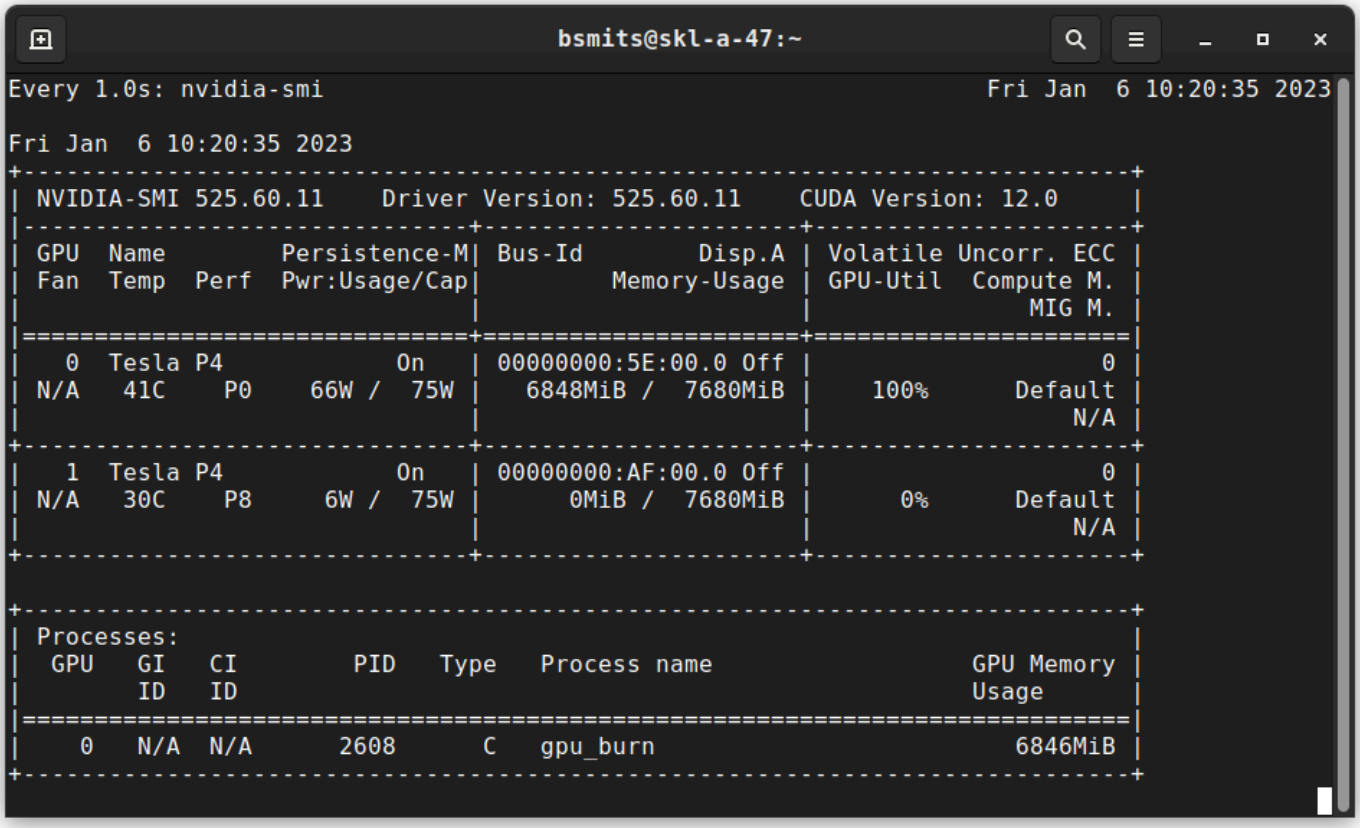


`nvidia-smi` shows us GPU usage (the `watch -n 1 <command>` just says re-run the `<command>` every second). At the bottom of `nvidia-smi`, we see that our `python3` process is using 518MB of GPU #0's memory. **Note:** GPU memory is **not** the same as the memory you requested in your sbatch script; each GPU has its own memory that automatically gets allocated to you when you ask Slurm for a GPU.

Note: GPUs have their own memory, which is independent of the RAM you asked for in your sbatch script.

In the middle of the `nvidia-smi` output, we see that the p4 GPU we requested is being 32% utilized (Note: The cluster no longer has p4 GPUs). For this example, we're not doing enough computation to fully utilize the GPU. That's okay because we are using some of the GPU. You want to make sure that if you ask for a GPU, you're at least partially using it. If the utilization is 0%, that tells you that you need to go over your code and try to figure out why it isn't using the GPU.

If we submit a different job that fully utilizes the GPU, we would see that in the output of `nvidia-smi` :



Note: You should use `nvidia-smi` to montior your job's GPU usage. 0% GPU? Check your code and figure out why it isn't using the GPU. 100% GPU? Make sure your code can leverage GPUs on multiple nodes before resubmitting with another GPU (and then check `nvidia-smi` to see if it's using both GPUs).

3 - Interactive Jobs

So far we have only talked about **batch** jobs; jobs that you submit and just let run until they finish. Our SPORC cluster was designed primarily for batch jobs, however not all computational workloads fit within that bucket.

For compute workloads that require you to interact with the code or a software application (*e.g.* MATLAB, Ansys), you can submit an interactive job.

Interactive jobs will schedule on the `interactive` partition, which has a maximum time limit of 12 hours. The `interactive` partition has 2 nodes with 56 CPUs each.

To submit an interactive job, use the command `sinteractive`:

1. `$ sinteractive`
2. You will be prompted to choose a Slurm account (your default account will be filled in). Type the name of your account and press Enter.
   - ```
     Account [swen-331,mistakes,rc-help]: rc-help
     ```
3. You will be prompted to select how many CPUs you need. Make a selection and press Enter.
   - ```
     Core count: 2
     ```
4. You will be prompted to select your maximum time limit. Make a selection and press Enter.
   - ```
     Run time [Max: 12:00:00]: 0-6:0:0
     ```
5. You will see the following output and you will be automatically logged into the node running your interactive job.
   - ```
     Submitting Job!
     srun: job 15312351 queued and waiting for resources
     srun: job 15312351 has been allocated resources
     ```
6. Once you see the output above, your terminal will automatically log into the node where your interactive jobs is running. From here, you can run your code and monitor it.
7. When you are done with your interactive job, make sure you run `exit` or `logout` to cancel your job and return back to the submit node.

Warning: DO NOT run `sbatch` or `srun` inside of an interactive session; if you do, you will confuse Slurm and your jobs may not behave the way you expect them to.

Note: You can also request GPUs for interactive jobs (`sinteractive --gres=gpu:<num_gpus>`), but be aware that GPUs on the interactive partition are limited.

Note: You can see your interactive jobs with `squeue` and other Slurm commands, just like a batch job.

Note: In order to launch an application that uses a Graphical User Interface (GUI), you need to use `ssh -X ...` to log into the cluster.

3.1.1 - Caveats for Windows

On Windows, your command prompt will ignore the `-X` flag for `ssh`, so you will need to download and use MobaXterm to connect to the cluster and launch `sinteractive` jobs.

4 - Slurm Command Reference

4.1 - sinfo

The `sinfo` command will tell you some useful information about the available partitions on the cluster, including a partition's time limit, how many nodes are available on that partition, which nodes are available on that partition, and the state of those nodes.

```
$ sinfo
PARTITION   AVAIL  TIMELIMIT  NODES  STATE NODELIST
tier3          up 5-00:00:00     61    mix skl-a-[01-46,49-61,64],theocho
tier3          up 5-00:00:00      2  alloc skl-a-[62-63]
debug          up 1-00:00:00      1    mix skl-a-47
debug          up 1-00:00:00      1   idle skl-a-48
interactive    up   12:00:00      1    mix clx-a-01
interactive    up   12:00:00      1   idle clx-a-02
```

If you want more specifics, you can use `sinfo -o '%11P %5D %22N %4c %21G %7m %11l'`:

```
$ sinfo -o '%11P %5D %22N %4c %21G %7m %11l'
PARTITION   NODES NODELIST               CPUS GRES                  MEMORY  TIMELIMIT
tier3       14    skl-a-[01,25,49-60]    36   gpu:a100:4            380000  20-00:00:00
tier3       21    skl-a-[26-46]          36   gpu:a100:2            340000+ 20-00:00:00
tier3       27    skl-a-[02-24,61-64]    36   (null)                380000  20-00:00:00
grace       2     gg-[00-01]             144  (null)                450000  20-00:00:00
grace       2     gh-[00-01]             72   gpu:gh200:1           450000  20-00:00:00
debug       1     skl-a-47               36   (null)                380000  1-00:00:00
debug       1     skl-a-48               36   gpu:a100:2            380000  1-00:00:00
onboard     2     clx-a-[01-02]          56   gpu:a100:1            340000+ 1:00:00
interactive 2     clx-a-[01-02]          56   gpu:a100:1            340000+ 12:00:00
```

Now we can see what kinds of GPUs (and how many) are available on each node. For example, `gpu:a100:4` means there are 4 a100 GPUs available on a node.

We can also see the maximum memory available on each node (in Megabytes).

With `sinfo -N` we can see details for each node (instead of grouping them by partition):

```
$ sinfo -N
NODELIST   NODES   PARTITION       STATE
clx-a-01       1 interactive       mixed
clx-a-02       1 interactive        idle
skl-a-01       1       tier3       mixed
skl-a-02       1       tier3       mixed
skl-a-03       1       tier3       mixed
...
skl-a-47       1       debug       mixed
skl-a-48       1       debug        idle
...
skl-a-62       1       tier3   allocated
skl-a-63       1       tier3   allocated
skl-a-64       1       tier3       mixed
theocho        1       tier3       mixed
```

In the above example, we see can see the **STATE** of each node. An `idle` state means none of the CPUs on a node are being used. `mixed` means some CPUs are being used and others are not. `allocated` means all of the CPUs on a node are being used.

4.2 - squeue

We already saw the `squeue` command, but it's okay to review. The `squeue` command will show you what jobs are currently scheduled:

```
$ squeue
       JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
    15308642     debug  1colony   bsmits  R   18:53:17      1 skl-a-47
    15311320 interacti _interac   ehdeec  R    2:47:40      1 clx-a-01
    15312325 interacti _interac   kmaits  R    1:24:59      1 clx-a-01
    15311322     tier3 jch2009_   jchits PD       0:00      1 (Resources)
    15308581     tier3 VR101_c1   slpits PD       0:00      1 (Priority)
    15308633     tier3 VR50_c10   slpits PD       0:00      1 (Priority)
    15308634     tier3 R101_c10   slpits PD       0:00      1 (Priority)
    ...
```

In the above example, we can see the job ID for each scheduled job, the partition each job is scheduled for, the name of the job, the user who submitted the job, the status (`ST`) of the job, how long the job has been running, how many nodes have been allocated to the job, and the nodes the job is scheduled to run on.

For jobs that are pending (`PD`), Slurm will give us a reason, which is typically **Resources** or **Priority**:

- `Priority` : When Slurm schedules a job, it takes into consideration your prior usage. If you often use a lot of resources, Slurm will assign your jobs a lower priority than someone who uses fewer resources than you, to ensure fair access to cluster resources for all researchers—you can read about the Fair Tree Fairshare Algorithm for details on how Slurm does this. Don't worry, your job will run eventually.
- `Resources` : Slurm is waiting for the requested reasources to be available before starting your job.

To see only your jobs, you can run `squeue --me` :

```
$ squeue --me
       JOBID PARTITION     NAME     USER  ST       TIME  NODES NODELIST(REASON)
    15311319     debug simple_m   abc1234   R       0:19      1 skl-a-47
    15311320     tier3 classifi   abc1234   R 1-08:27:13      2 skl-a-[01-02]
    15311321     tier3 normaliz   abc1234 PD       0:00      1 (Priority)
    15311322     tier3 nlp_word   abc1234 PD       0:00     10 (Resources)
```

If you want to know when your job will start, you can use the `--start` flag with `squeue` :

```
$ squeue --me --start
       JOBID PARTITION     NAME     USER ST          START_TIME  NODES SCHEDNODES
NODELIST(REASON)
    15312338     debug simple_g  abc1234 PD                 N/A      1 (null)
(Resources)
    15312338     debug simple_g  abc1234 PD 2022-10-27T17:33:31      1 skl-a-47
(Resources)
```

If `START_TIME` is `N/A` , that means Slurm is still figuring out the best way to schedule your job. Wait a few minutes and you should see a start time. **Note:** The start time shown here is a **Worst Case** scenario, meaning if no jobs finish early or get canceled, your job will start running around that time. However, it is likely that your job will start sooner.

4.3 - sbatch

You can use the `sbatch` command to submit your jobs:

```
$ sbatch slurm_script.sh
Submitted batch job 15312335
```

4.4 - scancel

If you submit a job by mistake, or notice an error in your code after submitting, you can use the `scancel <job_id>` command to cancel your job:

```
$ scancel 15304057
```

Note that `scancel` will not give you any output.

If you wish to cancel all of your jobs, you can use `scancel --me`.

4.5 - sacct

If you lose track of your recent jobs, you can use `sacct` to find their job IDs and a few other details:

```
$ sacct
JobID           JobName  Partition    Account  AllocCPUS      State ExitCode
------------ ---------- ---------- ---------- ---------- ---------- --------
15301081     TAPB-PDA_+      tier3    rc-help          4    RUNNING      0:0
15301081.ba+      batch              rc-help          4    RUNNING      0:0
15301081.ex+     extern              rc-help          4    RUNNING      0:0
...
15311319     simple_mpi      debug    rc-help          4  COMPLETED      0:0
15311319.ba+      batch              rc-help          4  COMPLETED      0:0
15311319.ex+     extern              rc-help          4  COMPLETED      0:0
15311319.0    hello.mpi              rc-help          4  COMPLETED      0:0
...
15311320     simple_mpi      debug    rc-help          4     FAILED      0:0
15311320.ba+      batch              rc-help          4     FAILED      0:0
15311320.ex+     extern              rc-help          4     FAILED      0:0
...
15304057        1colony      tier3    rc-help         44 CANCELLED+      0:0
...
15303993     TAPB-PDA_+      tier3    rc-help          8    PENDING      0:0
15303994     TAPB-PDA_+      tier3    rc-help          8    PENDING      0:0
15303995     TAPB-PDA_+      tier3    rc-help          8    PENDING      0:0
```

To view a specific job, run `sacct --jobs=<job_id>`:

```
$ sacct --jobs=15301081
JobID           JobName  Partition    Account  AllocCPUS      State ExitCode
------------ ---------- ---------- ---------- ---------- ---------- --------
15301081     TAPB-PDA_+      tier3    rc-help          4    RUNNING      0:0
15301081.ba+      batch               rc-help          4    RUNNING      0:0
15301081.ex+     extern               rc-help          4    RUNNING      0:0
```

You can also choose what information you want to see using the `--format` flag for `sacct`:

```
$ sacct --jobs=15301081 --format JobId,JobName,NCPUS,NNodes,ReqCPUs,ReqMem
JobID           JobName      NCPUS   NNodes  ReqCPUS     ReqMem
------------ ---------- ---------- -------- -------- ----------
15423809     3hr6_Osc_+          3        1        3       500G
15423809.ba+      batch          3        1        3
15423809.ex+     extern          3        1        3
```

In the above example, we are asking `sacct` to show us the Job ID (`JobId`), Job Name (`JobName`), number of allocated CPUs (`NCPUS`), number of allocated nodes (`NNodes`), number of requested CPUs (`ReqCPUs`), and amount of memory requested (`ReqMem`).

There are many fields we can pass to the `--format` flag to see details about a job. Here is a full list of job accounting fields.

Note: Some fields will not show anything until ***after a job has completed***. For example, the maximum memory used by a job (`MaxRSS`) will not populate until after a job is complete. The `MaxRSS` field will tell you how much memory your job used, which you can use to adjust your sbatch script.

4.6 - scontrol

The `scontrol` command has a lot of features, but the one you may find useful is `scontrol show job <job_id>`:

```
$ scontrol show job 15312335
JobId=15312336 JobName=simple_gpu
   UserId=bsmits(2894513) GroupId=staff(5001) MCS_label=N/A
   Priority=1998475 Nice=0 Account=rc-help QOS=qos_onboard
   JobState=RUNNING Reason=None Dependency=(null)
   Requeue=1 Restarts=0 BatchFlag=1 Reboot=0 ExitCode=0:0
   RunTime=00:00:07 TimeLimit=00:10:00 TimeMin=N/A
   SubmitTime=2022-10-27T12:52:47 EligibleTime=2022-10-27T12:52:47
   AccrueTime=2022-10-27T12:52:47
   StartTime=2022-10-27T12:52:48 EndTime=2022-10-27T13:02:48 Deadline=N/A
   SuspendTime=None SecsPreSuspend=0 LastSchedEval=2022-10-27T12:52:48 Scheduler=Main
   Partition=debug AllocNode:Sid=sporcbuild:10022
   ReqNodeList=(null) ExcNodeList=(null)
   NodeList=skl-a-47
   BatchHost=skl-a-47
   NumNodes=1 NumCPUs=1 NumTasks=1 CPUs/Task=1 ReqB:S:C:T=0:0:*:*
   TRES=cpu=1,mem=1G,node=1,billing=1,gres/gpu=1
   Socks/Node=* NtasksPerN:B:S:C=0:0:*:* CoreSpec=*
   MinCPUsNode=1 MinMemoryNode=1G MinTmpDiskNode=0
   Features=(null) DelayBoot=00:00:00
   OverSubscribe=OK Contiguous=0 Licenses=(null) Network=(null)
   Command=/home/bsmits/git_repos/example-scripts/10_simple_gpu/simple_gpu.sh
   WorkDir=/home/bsmits/git_repos/example-scripts/10_simple_gpu
   StdErr=/home/bsmits/git_repos/example-scripts/10_simple_gpu/simple_gpu_15312336.err
   StdIn=/dev/null
   StdOut=/home/bsmits/git_repos/example-scripts/10_simple_gpu/simple_gpu_15312336.out
   Power=
   TresPerNode=gres:gpu:a100:1
```

This will show you a lot of information. Here are some of the lines you may find useful:

- `JobState` : The state of your job. Usually `RUNNING` , `PENDING` , `COMPLETED` , or `FAILED` .
- `StartTime/EndTime` : What time your job started and when it will end.
- `Command` : The absolute path to the submission script for the job.
- `WorkDir` : The working directory for the job.
- `StdErr` : The absolute path to the location of your job's error file.
- `StdOut` : The absolute path to the location of your job's output file.

4.7 - my-accounts

The `my-accounts` command is not part of Slurm, but we created it to help you find out which Slurm accounts you have access to:

```
$ my-accounts
  Account Name     Expired  QOS              Allowed Partitions
- ------------     -------  ---              ------------------
  swen-331         true     qos_ood          ood
  mistakes         false    qos_tier3        interactive
* rc-help          false    qos_onboard      tier3,debug,onboard,interactive
```

4.8 - sinteractive

The `sinteractive` command will prompt you for an account, number of CPUS, and a time limit. After you enter those details, a new Slurm job is created, which allows you to interact with your code and/or launch software that has a Graphical User Interface (GUI).

```
abc1234@sporcsubmit ~/ $ sinteractive
Account [swen-331,mistakes,rc-help]: rc-help
Core count: 2
Run time [Max: 12:00:00]: 0-6:0:0

Submitting Job!

srun: job 15312351 queued and waiting for resources
srun: job 15312351 has been allocated resources

abc1234@clx-a-01 ~/ $
```

4.9 - sprio

One of the most common questions we get asked is *when will my job start?*. The quick answer is `squeue --me --start`, but that only tells you *when* your job will start, not *why* it will start at that time.

The `sprio` command lets you see which factors are impacting a job's scheduling priority. Running `sprio -l -S '-y'` will sort the queue by priority order and show scheduling priority factors for all jobs:

```
$ sprio -l -S '-y'
          JOBID PARTITION     USER  ACCOUNT   PRIORITY       SITE        AGE      ASSOC
FAIRSHARE    JOBSIZE  PARTITION    QOSNAME        QOS        NICE                 TRES
       20298200 tier3      jtr1801 cosmos-w     820494           0     134310           0
81369          0     604800  qos_tier3           0           0               cpu=15
       20299680 tier3       ss3105  iou-bce     722993           0      40626           0
77523          0     604800  qos_tier3           0           0    cpu=4,gres/gpu=40
       20299710 tier3       xy3371 playback     683987           0      38057           0
41089          0     604800  qos_tier3           0           0    cpu=1,gres/gpu=40
       20299712 tier3       xy3371 playback     683987           0      38057           0
41089          0     604800  qos_tier3           0           0    cpu=1,gres/gpu=40
       20300474 interacti   zs9580 rarl-ott     670147           0      20761           0
44328          0     604800  qos_tier3           0           0   cpu=9,gres/gpu=250
       20300980 ood         pp5291 proteinl     667622           0       7124           0
55663          0     604800  qos_tier3           0           0               cpu=36
       20299767 ood        tjr3717      ppi     666069           0      38186           0
23075          0     604800  qos_tier3           0           0                cpu=9
       20299670 ood         am2552   defake     665445           0      40857           0
19431          0     604800  qos_tier3           0           0 cpu=107,gres/gpu=250
       20229234 tier3       as7268      ppi     664199           0      36976           0
22265          0     604800  qos_tier3           0           0   cpu=6,gres/gpu=152
       20299749 tier3       bwbics   defake     662806           0      38766           0
19229          0     604800  qos_tier3           0           0    cpu=1,gres/gpu=10
       20301006 interacti   zs9580 rarl-ott     655864           0       6478           0
44328          0     604800  qos_tier3           0           0   cpu=9,gres/gpu=250
       20299121 tier3       ma7684      los     655832           0       4242           0
46757          0     604800  qos_tier3           0           0   cpu=14,gres/gpu=20
       20298878 interacti   zs9580 rarl-ott     649386           0          0           0
44328          0     604800  qos_tier3           0           0   cpu=9,gres/gpu=250
       20301238 tier3       zs9580 rarl-ott     649138           0          0           0
44328          0     604800  qos_tier3           0           0    cpu=0,gres/gpu=10
       20301236 tier3       zs9580 rarl-ott     649138           0          0           0
44328          0     604800  qos_tier3           0           0    cpu=0,gres/gpu=10
       20298784 tier3       xl3439   defake     640859           0      17022           0
19027          0     604800  qos_tier3           0           0    cpu=0,gres/gpu=10
       20301213 tier3       pb5785      ppi     628564           0       1021           0
22670          0     604800  qos_tier3           0           0    cpu=3,gres/gpu=71
       20301263 tier3       pb5785      ppi     627669           0        126           0
22670          0     604800  qos_tier3           0           0    cpu=3,gres/gpu=71
       20298786 tier3       xl3439   defake     625660           0       1823           0
19027          0     604800  qos_tier3           0           0    cpu=0,gres/gpu=10
       20298788 tier3       xl3439   defake     623837           0          0           0
19027          0     604800  qos_tier3           0           0    cpu=0,gres/gpu=10
       20298806 tier3       xl3439   defake     623837           0          0           0
19027          0     604800  qos_tier3           0           0    cpu=0,gres/gpu=10
       20298884 tier3       xl3439   defake     623837           0          0           0
19027          0     604800  qos_tier3           0           0    cpu=0,gres/gpu=10
       20293190 tier3       jk7877 2dptheor     616550           0       9702           0
2024           0     604800  qos_tier3           0           0               cpu=24
       20301226 tier3       crrvcs    stamp     614547           0        835           0
8906           0     604800  qos_tier3           0           0                cpu=7
       20301139 tier3       ia3494    tumor     614256           0       2969           0
```

```
6477          0     604800  qos_tier3           0            0    cpu=0,gres/gpu=10
       20301261 tier3       crrvcs    stamp     613900          0        188          0
8906          0     604800  qos_tier3           0            0                 cpu=7
       20298955 tier3       crrvcs    stamp     613712          0          0          0
8906          0     604800  qos_tier3           0            0                 cpu=7
       20298962 tier3       crrvcs    stamp     613712          0          0          0
8906          0     604800  qos_tier3           0            0                 cpu=7
       20301225 tier3       ia3494    tumor     612122          0        835          0
6477          0     604800  qos_tier3           0            0    cpu=0,gres/gpu=10
       20263735 tier3       ia3494    tumor     611287          0          0          0
6477          0     604800  qos_tier3           0            0    cpu=0,gres/gpu=10
       20226489 tier3       ia3494    tumor     611277          0          0          0
6477          0     604800  qos_tier3           0            0                 cpu=0
       20286809 tier3       bkecis  live-eo     610673          0          0          0
5870          0     604800  qos_tier3           0            0                 cpu=4
       20286810 tier3       bkecis  live-eo     610673          0          0          0
5870          0     604800  qos_tier3           0            0                 cpu=4
       ...
```

If you want to see the priority for a specific job, you can use `sprio --jobs <job_id>`:

```
$ sprio --jobs 15428486
      JOBID PARTITION   PRIORITY       SITE  FAIRSHARE  PARTITION        QOS
TRES
   15428486 tier3        1800760          0    1195952     604800          0     cpu=2,g
res/gpu=6
```

You can also show only your jobs with `sprio -u <username>`.

Now, lets talk about what those fields mean:

- `JOBID`: Job ID.
- `PARTITION`: Partition the job will run on. Partition does not impact priority at this time.
- `PRIORITY`: This is a number representing a job's priority relative to all other jobs. Typically, higher priority jobs will run before lower priority jobs, but this isn't *always* the case.
- `FAIRSHARE`: This is one of the factors that Slurm considers when scheduling jobs. It gets complicated, but basically this is a measure of how much a researcher and/or a Slurm account has been using the cluster recently. The more you use the cluster, the lower your `FAIRSHARE` value; this is to ensure that researchers just starting to use the cluster can get going quickly.
- `TRES`: This shows the number of CPUs and GPUs requested for a job. CPUs and GPUs have the same priority, but the number of CPUs/GPUs you request has a small (typically unnoticable) impact on priority. However, if you request a large number of CPUs/GPUs, that can still impact when your job schedules because Slurm has to wait for those resources to be available for your job.
- You can ignore `SITE` and `QOS`.

5 - Other Ways to Submit Jobs

5.1 - Open OnDemand

We use Open OnDemand as a web-based portal for the cluster. From OnDemand, you can view your home directory, monitor your jobs, launch Jupyter Notebooks, launch terminals, and launch desktop sessions.

Desktop sessions are particularly useful for running interactive sessions where you need to use a Graphical User Interface (GUI).

For more details, take a look at our OnDemand Documentation (using_open_ondemand.html).

6 - Conclusions

In this tutorial, you learned:

- That Slurm is a resource manager and job scheduler for compute clusters.
- How to tell Slurm what resources you need with an sbatch script.
- How to see what Slurm accounts you have access to (`my-accounts`).
- How to see details about the cluster nodes and partitions (`sinfo`).
- How to submit batch (`sbatch`) and interactive (`sinteractive`) jobs.
- How to cancel jobs (`scancel`).
- How to see what jobs are queued (`squeue`).
- How to monitor your jobs (`squeue` , `sacct` , `scontrol` , `sprio`).
- How to identify the resources you need using `htop` and `nvidia-smi` .

You also learned the importance of **requesting only the resources that you need** and when to use the **debug** partition vs. when to use **tier3**.

In Part 2 of this tutorial (slurm_tutorial_2.html), you will learn how to make the most of the cluster by parallelizing your job.

Slurm Tutorial Part 2: Scaling Up (slurm_tutorial_2.html)

Help: If there are any further questions, or there is an issue with the documentation, please **submit a ticket** or contact us on Slack for additional assistance.

Tags: instructions (tag_instructions.html) slurm (tag_slurm.html)

Internal Documentation

Page last updated: May 1, 2023

Site last generated: Mar 19, 2026